

DESPLIEGUE DE UNA APLICACIÓN MULTICAPA EN DOCKER SOBRE LINUX

APRENDIZ:

LAURA VANESSA PEREZ PERDOMO

INSTRUCTOR:

CARLOS JULIO CADENA

PROGRAMA:

ANALISIS Y DESARROLLO DE SOFTWARE

3145555

CENTRO DE FORMACION LA INDUSTRIA, LA EMPRESA Y LOS SERVICIOS

SENA SERVICIO DE APRENDIZAJE

NEIVA – HUILA

2026

Despliegue de una aplicación multicapa en Docker sobre Linux

Objetivo

Implementar y desplegar una aplicación multicapa dentro de una máquina virtual con Linux, utilizando Docker como plataforma de contenedores. Se seleccionan libremente las tecnologías para el frontend, backend y base de datos, garantizando su compatibilidad, su ejecución mediante Docker y su funcionamiento integrado.

Actividad 1 — Selección de la arquitectura tecnológica

Selección tecnológica y Justificación: Para este despliegue se ha elegido una arquitectura clásica de tres capas altamente eficiente:

Frontend (HTML/CSS/JavaScript + Nginx): Se utiliza JavaScript nativo (Vanilla JS) servido por Nginx por su extrema ligereza, rapidez de carga y porque no requiere dependencias pesadas, lo que hace que la imagen Docker sea mínima.

Backend (Java + Spring Boot): Se selecciona Spring Boot porque es un framework robusto, escalable y con tipado estático, ideal para crear APIs REST seguras a nivel empresarial.

Base de Datos (PostgreSQL): Se elige PostgreSQL por ser uno de los motores relacionales más avanzados de código abierto, con excelente rendimiento e integridad de datos. Además, se integra de forma nativa con herramientas como Liquibase para el control de versiones de la base de datos.

Tabla de Arquitectura:

Componente	Tecnología	Puerto	Función
Frontend	HTML/JS (Nginx)	80	Interfaz de usuario
Backend	Java + Spring Boot	8080	API REST
Base de datos	PostgreSQL 17	5432	Persistencia
Migraciones	Liquibase	N/A	Gestión de BD

Actividad 2 — Preparación de la máquina virtual Linux

Verificar: bash, hostname, hostnamectl, uname -a, free -h, df -h

```

laura@ubuntuuserver:~$ hostname
ubuntuuserver
laura@ubuntuuserver:~$ hostnamectl
Static hostname: ubuntuuserver
    Icon name: computer-vm
    Chassis: vm
    Machine ID: 404c37ddc7634647af367103297a9ed0
    Boot ID: ae4794c5e2e848c09255bbb64d44380e
    Virtualization: oracle
    Operating System: Ubuntu 26.04 LTS
    Kernel: Linux 7.0.0-30-generic
    Architecture: x86-64
    Hardware Vendor: innotek GmbH
    Hardware Model: VirtualBox
    Hardware Version: 1.2
    Firmware Version: VirtualBox
    Firmware Date: Fri 2006-12-01
    Firmware Age: 19y 8month 4w
laura@ubuntuuserver:~$ uname -a
Linux ubuntuuserver 7.0.0-30-generic #30-Ubuntu SMP PREEMPT_DYNAMIC Fri Jul 31 18:22:54 UTC 2026 x86_64 GNU/Linux
laura@ubuntuuserver:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:           1,6Gi         439Mi        542Mi         1,3Mi         811Mi         1,2Gi
Swap:           2,0Gi          12Mi         2,0Gi
laura@ubuntuuserver:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs            329M  1,3M  328M   1% /run
/dev/mapper/ubun--vg-ubuntu--lv 12G   8,7G   2,0G  82% /
tmpfs            822M   0 822M   0% /dev/shm
none            1,0M   0 1,0M   0% /run/credentials/systemd-journald.service
tmpfs            822M   0 822M   0% /tmp
none            1,0M   0 1,0M   0% /run/credentials/systemd-resolved.service
/dev/sda2        2,0G  183M   1,7G  10% /boot
none            1,0M   0 1,0M   0% /run/credentials/getty@tty1.service
tmpfs            165M   8,0K  165M   1% /run/user/1000
none            1,0M   0 1,0M   0% /run/credentials/systemd-networkd.service
laura@ubuntuuserver:~$

```

Descripción: Se verifica el estado y recursos de la máquina virtual ubuntuuserver con sistema operativo Ubuntu 26.04 LTS. Cuenta con 1.2 GB de memoria RAM disponible para correr los contenedores. Adicionalmente, el almacenamiento del disco principal fue optimizado y cuenta con 2.0 GB de espacio libre (82% en uso), suficiente para la compilación y construcción de imágenes Docker.

Verificar la red comandos: ip addr y ip route

```

laura@ubuntuuserver:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:6d:1c:a7 brd ff:ff:ff:ff:ff:ff
    altname enx0800276d1ca7
    inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 86026sec preferred_lft 86026sec
    inet6 fd17:625c:f037:2:a00:27ff:fe6d:1ca7/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86068sec preferred_lft 14068sec
    inet6 fe80::a00:27ff:fe6d:1ca7/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
3: br-d6d2d37c61f8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 8a:68:f9:04:e8:ef brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.1/16 brd 172.18.255.255 scope global br-d6d2d37c61f8
        valid_lft forever preferred_lft forever
    inet6 fe80::8868:f9ff:fe04:e8ef/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
4: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether fa:b2:3d:db:75:19 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
    inet6 fe80::f8b2:3dff:fedb:7519/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
5: br-221cce8856ed: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether c2:b3:dd:e6:3f:94 brd ff:ff:ff:ff:ff:ff
    inet 172.19.0.1/16 brd 172.19.255.255 scope global br-221cce8856ed
        valid_lft forever preferred_lft forever
6: br-77bef0c17437: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 6a:bd:16:ac:a9:1a brd ff:ff:ff:ff:ff:ff
    inet 172.20.0.1/16 brd 172.20.255.255 scope global br-77bef0c17437
        valid_lft forever preferred_lft forever
7: veth95c527b0if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-d6d2d37c61f8 state UP group default
    link/ether b6:2e:5b:a2:5e:75 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet6 fe80::b42e:5bff:fea2:5e75/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
8: veth57648ef0if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default

```

```

link/ether 06:05:a7:20:9c:25 brd ff:ff:ff:ff:ff:ff link-netnsid 1
inet6 fe80::405:a7ff:fe20:9c25/64 scope link proto kernel_ll
valid_lft forever preferred_lft forever
laura@ubuntu:~$ ip route
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.0.2.2 dev enp0s3 proto dhcp scope link src 10.0.2.15 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1
172.18.0.0/16 dev br-d6d2d37c61f8 proto kernel scope link src 172.18.0.1
172.19.0.0/16 dev br-221cce8856ed proto kernel scope link src 172.19.0.1 linkdown
172.20.0.0/16 dev br-77bef0c17437 proto kernel scope link src 172.20.0.1 linkdown
200.21.200.0 via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
200.21.200.80 via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
laura@ubuntu:~$

```

Descripción: Se verifica la configuración de red y el enrutamiento del servidor virtual. A través de `ip addr` se identifica la IP privada principal 10.0.2.15 asignada a la interfaz `enp0s3`. Mediante `ip route` se confirma la puerta de enlace predeterminada 10.0.2.2. También se listan las interfaces de red creadas por los entornos de Docker (`docker0` y puentes virtuales).

Comprobar conectividad comandos: `ping 8.8.8.8`

```

laura@ubuntu:~$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=64 time=15.0 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=64 time=16.0 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=64 time=16.0 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=64 time=45.8 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3035ms
rtt min/avg/max/mdev = 15.003/23.209/45.794/13.045 ms
laura@ubuntu:~$

```

Descripción: Se realiza una prueba de conexión a los DNS públicos de Google (8.8.8.8) transmitiendo 4 paquetes. La prueba arroja un resultado exitoso de 0% packet loss (cero paquetes perdidos) con tiempos de respuesta óptimos, validando la salida activa del servidor hacia Internet.

Actividad 3 — Instalación y configuración de Docker

Comando a ejecutar: `docker --version` y `docker compose version`

Comprobar el servicio: `sudo systemctl status docker`

```

laura@ubuntuuserver:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
laura@ubuntuuserver:~$ docker compose version
Docker Compose version 2.40.3+ds1-0ubuntu1
laura@ubuntuuserver:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-08-30 04:06:20 UTC; 15min ago
     Invocation: ac0c89311e0b4e9a93da944e5e8e5ef5
   TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 1225 (dockerd)
      Tasks: 54
     Memory: 128.1M (peak: 130.6M)
        CPU: 16.679s
    CGroup: /system.slice/docker.service
            └─1225 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
              └─1752 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8000 -container-ip 172.17.0.2
                └─1762 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8000 -container-ip 172.17.0.2
                  └─1776 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 9443 -container-ip 172.17.0.2
                    └─1782 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 9443 -container-ip 172.17.0.2
                      └─1819 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8081 -container-ip 172.17.0.2
                        └─1826 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8081 -container-ip 172.17.0.2

ago 30 04:06:15 ubuntuuserver dockerd[1225]: time="2026-08-30T04:06:15.591112717Z" level=warning msg="WARNING: The daemon is running in insecure mode, which is deprecated and will be removed in a future release."
ago 30 04:06:17 ubuntuuserver dockerd[1225]: time="2026-08-30T04:06:17.856286704Z" level=info msg="Starting daemon"
ago 30 04:06:18 ubuntuuserver dockerd[1225]: time="2026-08-30T04:06:18.408699313Z" level=info msg="Setting up the daemon"
ago 30 04:06:19 ubuntuuserver dockerd[1225]: time="2026-08-30T04:06:19.424961595Z" level=info msg="Listening for connections"
ago 30 04:06:19 ubuntuuserver dockerd[1225]: time="2026-08-30T04:06:19.590170046Z" level=info msg="Daemon is ready"
ago 30 04:06:19 ubuntuuserver dockerd[1225]: time="2026-08-30T04:06:19.593511379Z" level=info msg="Starting daemon"
ago 30 04:06:19 ubuntuuserver dockerd[1225]: time="2026-08-30T04:06:19.876144311Z" level=info msg="Setting up the daemon"
ago 30 04:06:20 ubuntuuserver dockerd[1225]: time="2026-08-30T04:06:20.045783649Z" level=info msg="Listening for connections"
ago 30 04:06:20 ubuntuuserver dockerd[1225]: time="2026-08-30T04:06:20.045865433Z" level=info msg="Daemon is ready"
ago 30 04:06:20 ubuntuuserver systemd[1]: Started docker.service - Docker Application Container Engine
laura@ubuntuuserver:~$

```

Descripción: Se comprueba el correcto funcionamiento de la plataforma de contenedores. Se muestra la instalación de Docker (versión 29.1.3) y Docker Compose (versión 2.40.3). Con el comando `systemctl status` se constata que el servicio de Docker está habilitado y en estado activo (active (running)), listo para gestionar contenedores en el sistema.

Actividad 4 — Primer contenedor

Ejecutar un contenedor de prueba: `docker run hello-world`

```
laura@ubuntu:~$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

laura@ubuntu:~$
```

Descripción: Se ejecuta con éxito el primer contenedor de prueba utilizando la imagen oficial hello-world desde Docker Hub. La salida en consola confirma que el cliente de Docker se conectó correctamente al demonio local (daemon) y que la instalación es operativa.

Posteriormente ejecutar un servidor web: `sudo docker run -d --name nginx-prueba -p 8080:80 nginx`
Comprobar: `docker ps`

```
laura@ubuntu:~$ docker run -d --name nginx-prueba -p 8080:80 nginx
9947317c074870eaa9b9d53cd3ee69e0e38f69fa36305ad62aee3c85f3b16e0
laura@ubuntu:~$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
9947317c0748	nginx	"/docker-entrypoint..."	14 seconds ago	Up 12 seconds
ds	0.0.0.0:8080->80/tcp	nginx-prueba		
30daaa0a97bd	eclipse-temurin:17-jdk-jammy	"/_cacert_entrypoint..."	4 days ago	Up 22 minutes
es	0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp	app-backend		
eed0ecfc7373	portainer/portainer-ce:latest	"/portainer"	4 days ago	Up 22 minutes
es	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp, 9000/tcp	portainer		

Descripción: Se valida el inicio del servidor web Nginx en segundo plano mediante `docker ps`. El contenedor `nginx-prueba` se encuentra en ejecución activa (Up 12 seconds) y con el puerto 8080 de la máquina virtual correctamente redireccionado al puerto 80 interno del contenedor.

Actividad 5 — Administración de contenedores

Realizar operaciones de:

`docker ps`

`docker ps -a`

docker stop

docker start

docker restart

docker logs

docker inspect

docker rm

```
laura@ubuntu:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        NAMES
aa5a702f7620   nginx     "/docker-entrypoint..." About a minute ago Up About a minute nginx-pr
0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
ueba
30daaa0a97bd   eclipse-temurin:17-jdk-jammy "/__cacert_entrypoint..." 4 days ago    Up 35 minutes app-back
0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
end
eed0ecfc7373   portainer/portainer-ce:latest "/portainer"        4 days ago    Up 35 minutes portaine
0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp, 9000/tcp
r
laura@ubuntu:~$ docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        NAMES
aa5a702f7620   nginx     "/docker-entrypoint..." About a minute ago Up About a minute
0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
nginx-prueba
92588f5e4971   hello-world "/hello"            14 minutes ago Exited (0) 14 minu
tes ago
relaxed_aryabhata
a248b4b8c72d   hello-world "/hello"            15 minutes ago Exited (0) 15 minu
tes ago
vigilant_mestorf
011278d7226b   cine-frontend-img    "/docker-entrypoint..." 4 days ago    Exited (255) 3 day
s ago
0.0.0.0:80->80/tcp, [::]:80->80/tcp
test-frontend
30daaa0a97bd   eclipse-temurin:17-jdk-jammy "/__cacert_entrypoint..." 4 days ago    Up 35 minutes
0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
app-backend
eed0ecfc7373   portainer/portainer-ce:latest "/portainer"        4 days ago    Up 35 minutes
0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp, 9000/tcp
portainer
57ccf27622ff   hello-world "/hello"            4 days ago    Exited (0) 4 days
ago
clever_rhodes
laura@ubuntu:~$ docker stop nginx-prueba
nginx-prueba
```

```

laura@buntuserver:~$ docker start nginx-prueba
nginx-prueba
laura@buntuserver:~$ docker restart nginx-prueba
nginx-prueba
laura@buntuserver:~$ docker logs nginx-prueba
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/08/30 04:40:13 [notice] 1#1: using the "epoll" event method
2026/08/30 04:40:13 [notice] 1#1: nginx/1.31.4
2026/08/30 04:40:13 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/08/30 04:40:13 [notice] 1#1: OS: Linux 7.0.0-30-generic
2026/08/30 04:40:13 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/08/30 04:40:13 [notice] 1#1: start worker processes
2026/08/30 04:40:13 [notice] 1#1: start worker process 29
2026/08/30 04:40:13 [notice] 1#1: start worker process 30
2026/08/30 04:41:35 [notice] 1#1: signal 3 (SIGQUIT) received, shutting down
2026/08/30 04:41:35 [notice] 29#29: gracefully shutting down
2026/08/30 04:41:35 [notice] 30#30: gracefully shutting down
2026/08/30 04:41:35 [notice] 29#29: exiting
2026/08/30 04:41:35 [notice] 30#30: exiting
2026/08/30 04:41:35 [notice] 30#30: exit
2026/08/30 04:41:35 [notice] 29#29: exit
2026/08/30 04:41:35 [notice] 1#1: signal 17 (SIGCHLD) received from 29
2026/08/30 04:41:35 [notice] 1#1: worker process 29 exited with code 0
2026/08/30 04:41:35 [notice] 1#1: signal 29 (SIGIO) received
2026/08/30 04:41:35 [notice] 1#1: signal 17 (SIGCHLD) received from 30
2026/08/30 04:41:35 [notice] 1#1: worker process 30 exited with code 0
2026/08/30 04:41:35 [notice] 1#1: exit
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: IPv6 listen already enabled
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/08/30 04:43:09 [notice] 1#1: using the "epoll" event method
2026/08/30 04:43:09 [notice] 1#1: nginx/1.31.4
2026/08/30 04:43:09 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/08/30 04:43:09 [notice] 1#1: OS: Linux 7.0.0-30-generic
2026/08/30 04:43:09 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/08/30 04:43:09 [notice] 1#1: start worker processes

```

```

2026/08/30 04:43:09 [notice] 1#1: start worker process 22
2026/08/30 04:43:09 [notice] 1#1: start worker process 23
2026/08/30 04:43:16 [notice] 1#1: signal 3 (SIGQUIT) received, shutting down
2026/08/30 04:43:16 [notice] 22#22: gracefully shutting down
2026/08/30 04:43:16 [notice] 22#22: exiting
2026/08/30 04:43:16 [notice] 22#22: exit
2026/08/30 04:43:16 [notice] 23#23: gracefully shutting down
2026/08/30 04:43:16 [notice] 23#23: exiting
2026/08/30 04:43:16 [notice] 23#23: exit
2026/08/30 04:43:16 [notice] 1#1: signal 17 (SIGCHLD) received from 23
2026/08/30 04:43:16 [notice] 1#1: worker process 23 exited with code 0
2026/08/30 04:43:16 [notice] 1#1: signal 29 (SIGIO) received
2026/08/30 04:43:16 [notice] 1#1: signal 17 (SIGCHLD) received from 22
2026/08/30 04:43:16 [notice] 1#1: worker process 22 exited with code 0
2026/08/30 04:43:16 [notice] 1#1: exit
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: IPv6 listen already enabled
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/08/30 04:43:18 [notice] 1#1: using the "epoll" event method
2026/08/30 04:43:18 [notice] 1#1: nginx/1.31.4
2026/08/30 04:43:18 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/08/30 04:43:18 [notice] 1#1: OS: Linux 7.0.0-30-generic
2026/08/30 04:43:18 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/08/30 04:43:18 [notice] 1#1: start worker processes
2026/08/30 04:43:18 [notice] 1#1: start worker process 22
2026/08/30 04:43:18 [notice] 1#1: start worker process 23

```

```

laura@buntuserver:~$ docker inspect nginx-prueba
[
  {
    "Id": "aa5a702f7620672afa08ae09289b981f799b6bf8b97fbee488c13cc50277c9f3",
    "Created": "2026-08-30T04:40:11.584729678Z",
    "Path": "/docker-entrypoint.sh",
    "Args": [
      "nginx",
      "-g",
      "daemon off;"
    ],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 4960,

```


PORTAINER.IO

New Portainer installation

Please create the initial administrator user.

Username

admin

Password

Confirm password

✓

Setup token ⓘ

22a51ad3e69d67c16d758f783b3eaf4d2501359eb08bd18bbb8ea25d3856688e

Find this token in the Portainer server logs. See the [setup token FAQ](#) for more information.

⚠ The password must be at least 12 characters long. ✓

Create user

> Restore Portainer from backup

Descripción: En esta captura se demuestra la correcta instalación y ejecución de Portainer mediante Docker. Gracias a la configuración de redirección de puertos, se logró acceder a la interfaz gráfica del administrador a través del navegador web del equipo anfitrión, cargando exitosamente la pantalla inicial de configuración de usuario.

Evidencia Administración del entorno Docker desde Portainer:

Portainer | local

No seguro https://127.0.0.1:9443/#/3/docker/dashboard

Upgrade to Business Edition

PORTAINER.IO
COMMUNITY EDITION

Home

local

Dashboard

Templates

Stacks

Containers

Images

Networks

Volumes

Events

Host

Administration

User-related

Environment-related

Registries

Logs

Notifications

Environment summary

Dashboard

Environment info

Environment

local - Standalone 29.1.3

URL

/var/run/docker.sock

Tags

-

0 Stacks

3 Containers

2 running 1 stopped
0 healthy 0 unhealthy

4 Images

811 MB

1 Volume

3 Networks

Descripción: En esta captura se evidencia el acceso exitoso al panel de administración gráfica de Portainer tras configurar la cuenta de administrador. Se seleccionó el entorno Docker local y se visualiza el Dashboard principal, el cual cumple con la visualización integral de los recursos del

sistema: confirmando la existencia de contenedores, imágenes descargadas, redes configuradas y volúmenes de almacenamiento.

Actividad 7 — Volúmenes Docker

Crear y utilizar volúmenes Docker: docker volume create datos_aplicacion

Comprobar: docker volume ls

El volumen deberá utilizarse para almacenar información que deba sobrevivir al ciclo de vida del contenedor.

Evidencia Volumen creado:

```
laura@ubuntu:~$ docker volume create datos_aplicacion
datos_aplicacion
laura@ubuntu:~$ docker volume ls
DRIVER      VOLUME NAME
local       229d1b90d2f42221c89b0793d1287910b14b429ea5a3aa594e344a5f0d2b7fc8
local       datos_aplicacion
local       flae7770b146cbcc9e9fb136012197726842016c7df6e68b63b1b855be483c26
local       laura_vol_datos
local       portainer_data
local       vol_basedatos
laura@ubuntu:~$
```

Descripción: Se ejecuta el comando docker volume create datos_aplicacion para crear un volumen nombrado local. Al realizar el listado con docker volume ls, se confirma la existencia y registro del volumen datos_aplicacion en el host.

Evidencia Prueba de persistencia:

```
laura@ubuntu:~$ docker run -d --name test-persistence -v datos_aplicacion:/data alpine sleep 3600
docker exec test-persistence sh -c "echo 'Datos Persistidos del Cine' > /data/prueba.txt"
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
56dceff11b33: Download complete
f5124fb579e2: Download complete
Digest: sha256:28bd5fe8b56d1bd048e5babf5b10710ebe0bae67db86916198a6eec434943f8b
Status: Downloaded newer image for alpine:latest
2117d734c93613ec2faa2cfe88ee63ad2c5f74e5942ba2c91926b3e6a7c821da
laura@ubuntu:~$ docker stop test-persistence
test-persistence
laura@ubuntu:~$ docker rm test-persistence
test-persistence
laura@ubuntu:~$ docker run --rm -v datos_aplicacion:/data alpine cat /data/prueba.txt
Datos Persistidos del Cine
laura@ubuntu:~$
```

Descripción: Se monta el volumen datos_aplicacion en el contenedor test-persistence y se crea un archivo de prueba. Tras detener y eliminar completamente el contenedor, se levanta uno nuevo mapeado al mismo volumen. Al leer el archivo en el nuevo contenedor, se recupera exitosamente

la cadena "Datos Persistidos del Cine", validando que los datos se almacenen fuera del ciclo de vida de los contenedores.

Actividad 8 — Dockerfile del frontend

Crear un Dockerfile para el frontend seleccionado. El Dockerfile deberá permitir:

1. Obtener la imagen base.
2. Instalar dependencias.
3. Copiar el código.
4. Construir la aplicación, cuando corresponda.
5. Ejecutar el servicio.
6. Exponer el puerto correspondiente.

Ejemplo conceptual:

Código fuente

|



Dockerfile

|



Imagen Docker

|



Contenedor Frontend

Evidencia Dockerfile del frontend:

```

laura@ubuntu-server:~/despliegue-multicapa-linux$ cat Frontend_Cine/cinema/Dockerfile
FROM nginx:alpine

# Copiar configuración personalizada de nginx
COPY nginx.conf /etc/nginx/conf.d/default.conf

# Copiar archivos del frontend al directorio de nginx
COPY . /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
laura@ubuntu-server:~/despliegue-multicapa-linux$

```

Descripción: Se visualiza el contenido del Dockerfile diseñado para el Frontend. Utiliza la imagen base ultraligera nginx:alpine, copia un archivo de configuración de Nginx personalizado (para actuar como proxy inverso), transfiere el código estático de la interfaz (HTML/JS/CSS) a la ruta de publicación del servidor, expone el puerto 80 y define el comando de ejecución en primer plano para Nginx.

Evidencia Imagen Docker construida:

```

laura@ubuntu-server:~/despliegue-multicapa-linux$ docker build -t cine-frontend-img ./Frontend_Cine/cinema
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 72.19kB
Step 1/5 : FROM nginx:alpine
----> db35bfc6b295
Step 2/5 : COPY nginx.conf /etc/nginx/conf.d/default.conf
----> Using cache
----> bcfadee88ab0
Step 3/5 : COPY . /usr/share/nginx/html
----> Using cache
----> b0d0e5adcc0b
Step 4/5 : EXPOSE 80
----> Using cache
----> b10607a65223
Step 5/5 : CMD ["nginx", "-g", "daemon off;"]
----> Using cache
----> 3d196ef6533f
Successfully built 3d196ef6533f
Successfully tagged cine-frontend-img:latest
laura@ubuntu-server:~/despliegue-multicapa-linux$ docker images

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
alpine:latest	28bd5fe8b56d	13MB	3.93MB	
cine-frontend-img:latest	3d196ef6533f	93.5MB	26.3MB	U
dpage/pgadmin4:latest	2f4ce946ddf8	778MB	182MB	
eclipse-temurin:17-jdk-jammy	400014962ad7	635MB	200MB	U
hello-world:latest	5dd0d3e6e255	25.9kB	9.49kB	U
mi-backend-app:latest	cd03f98afadb	196MB	47.3MB	
mi-backend-real:latest	12b9e4b9c5b4	199MB	47.9MB	
mi-frontend-app:latest	fdb57f71c596	196MB	47.3MB	
nginx:alpine	db35bfc6b295	94.2MB	27.2MB	
nginx:latest	b34848eff6db	241MB	66.2MB	
node:18-alpine	8d6421d663b4	181MB	45.3MB	
portainer/portainer-ce:2.19.4	908d04d20e86	382MB	88.1MB	
portainer/portainer-ce:latest	3fa8750ac2b9	188MB	42MB	U
postgres:17	0b657ff48d7f	645MB	167MB	

```

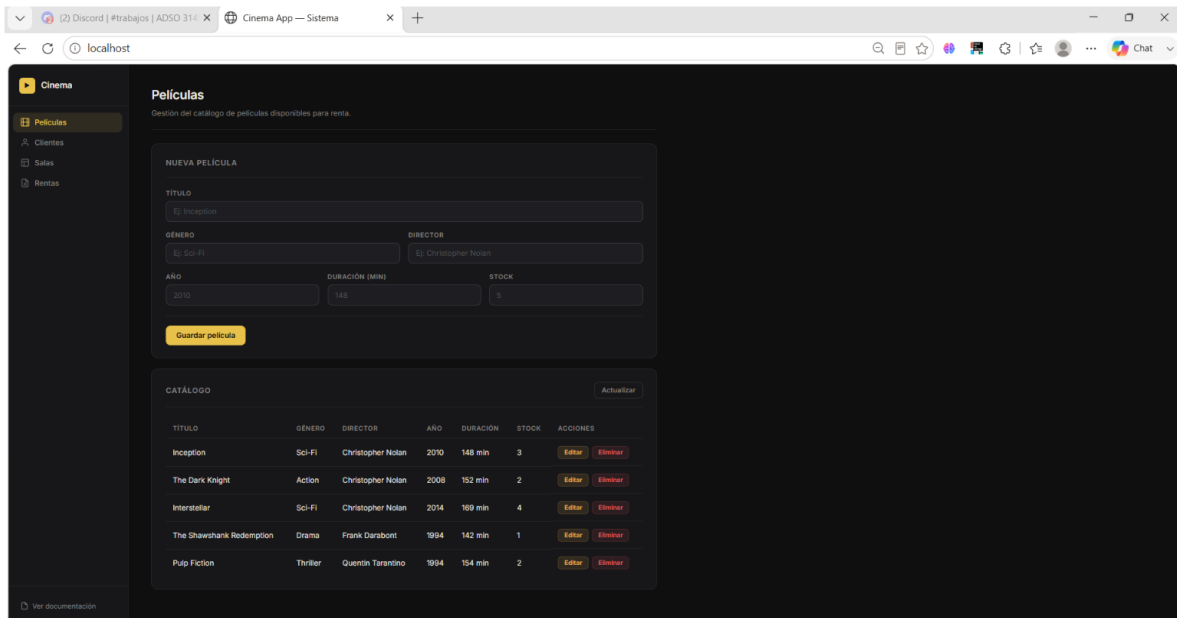
laura@ubuntu-server:~/despliegue-multicapa-linux$

```

Descripción: Se compila la imagen Docker del Frontend mediante el comando docker build asignándole el tag cine-frontend-img:latest. Al listar las imágenes locales con docker images, se comprueba que fue creada correctamente con un peso optimizado de tan solo 93.5 MB, gracias al uso de la imagen base Alpine de Nginx.

Evidencia Frontend funcionando dentro del contenedor:

```
laura@ubuntu-server:~/despliegue-multicapa-linux$ docker run -d --name test-frontend -p 80:80 cine-frontend-img
067a8460438b66f3901a90588817ff15a0659128a3cd1cfff6ed442a43f147846
laura@ubuntu-server:~/despliegue-multicapa-linux$
```



Descripción: Se demuestra el funcionamiento del Frontend accediendo desde el navegador de Windows a `http://localhost`. Se observa la interfaz web activa del catálogo de películas del cine, la cual renderiza de forma exitosa los datos obtenidos de la API del backend, validando que el servidor Nginx y el enrutamiento están totalmente operativos.

Actividad 9 — Dockerfile del backend

Crear un Dockerfile para el backend, el contenedor deberá:

- * Ejecutar el backend.
- * Exponer el puerto correspondiente.
- * Permitir recibir solicitudes del frontend.
- * Permitir comunicarse con la base de datos.
- * Utilizar variables de entorno para la configuración.

Evidencia Dockerfile del backend:

```

laura@ubuntu:~/despliegue-multicapa-linux$ cat Backend_API_Cine/cinema/Dockerfile
# ---- Etapa 1: compilar ----
FROM maven:3.9.6-eclipse-temurin-17 AS builder

WORKDIR /app

# Copiar primero solo el pom.xml para aprovechar caché de dependencias
COPY pom.xml .
RUN mvn dependency:go-offline -B

# Copiar el código fuente y compilar
COPY src ./src
RUN mvn clean package -DskipTests -B

# ---- Etapa 2: imagen final ligera ----
FROM eclipse-temurin:17-jre-alpine

WORKDIR /app

# Copiar el JAR generado en la etapa anterior
COPY --from=builder /app/target/*.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
laura@ubuntu:~/despliegue-multicapa-linux$

```

Descripción: Se visualiza el contenido del Dockerfile multi-etapa (multi-stage build) del Backend. En la Etapa 1 (builder) se utiliza la imagen base de maven para descargar las dependencias de Java y compilar la aplicación Spring Boot generando el archivo .jar. En la Etapa 2 (final) se utiliza una imagen JRE de eclipse-temurin basada en Alpine (altamente ligera), se copia el archivo empaquetado .jar, se expone el puerto 8080 y se define el punto de entrada para levantar la API REST.

Evidencia Imagen del backend:

```

laura@ubuntu:~/despliegue-multicapa-linux$ docker build -t cine-backend-img ./Backend_API_Cine/cinema
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 387.1kB
Step 1/11 : FROM maven:3.9.6-eclipse-temurin-17 AS builder
--> 29a1658b1f30
Step 2/11 : WORKDIR /app
--> Using cache
--> b4c882ae88a2
Step 3/11 : COPY pom.xml .
--> Using cache
--> b618b2134032
Step 4/11 : RUN mvn dependency:go-offline -B
--> Using cache
--> 4a40dc407bd1
Step 5/11 : COPY src ./src
--> Using cache
--> b9039b35a409
Step 6/11 : RUN mvn clean package -DskipTests -B
--> Using cache
--> c2dc770ec604
Step 7/11 : FROM eclipse-temurin:17-jre-alpine
17-jre-alpine: Pulling from library/eclipse-temurin
e01da199f243: Pulling fs layer
444cc034512a4: Pulling fs layer
8833a79c92c1: Pulling fs layer
Digest: sha256:27cc0849148c0fd32ee8e95988917becf9bc96a3182a24f99d9763aa8e90f8cb
Status: Downloaded newer image for eclipse-temurin:17-jre-alpine
--> 27cc0849148c
Step 8/11 : WORKDIR /app
--> Running in 08beb4d42f37
--> Removed intermediate container 08beb4d42f37
--> b72aafa25b6f
Step 9/11 : COPY --from=builder /app/target/*.jar app.jar
--> f4971b1adb1b
Step 10/11 : EXPOSE 8080
--> Running in 36ec179a39a9
--> Removed intermediate container 36ec179a39a9
--> ac938b699261
Step 11/11 : ENTRYPOINT ["java", "-jar", "app.jar"]
--> Running in dfc23634310e
--> Removed intermediate container dfc23634310e
--> 8ae6810126f9

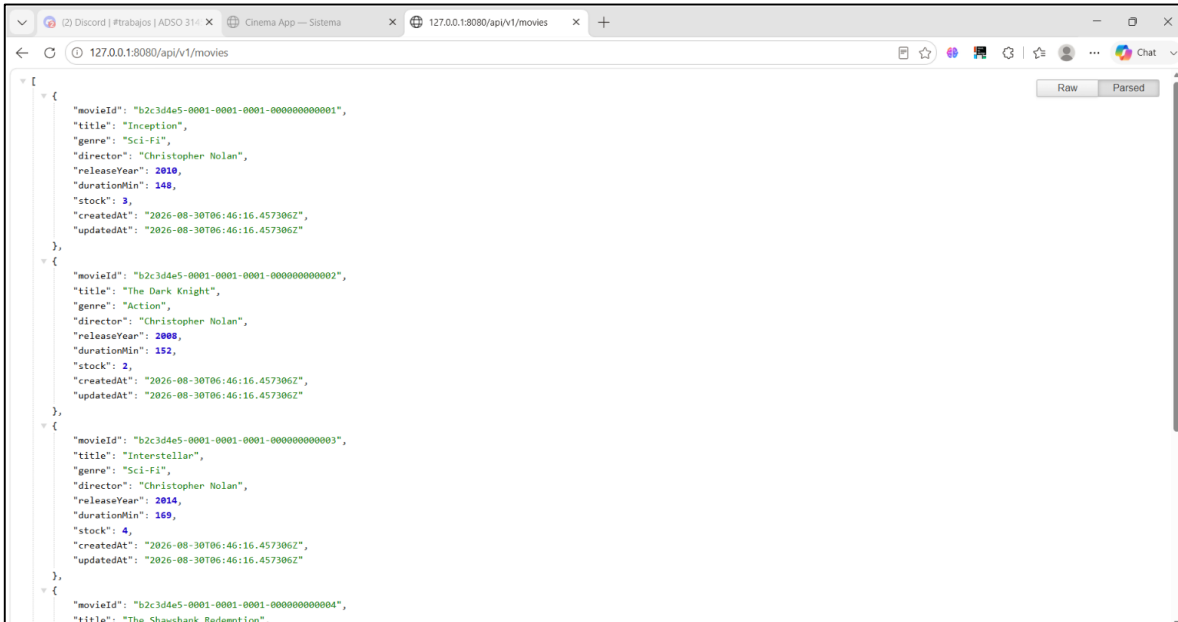
```

```
---> 8ae6810126f9
Successfully built 8ae6810126f9
Successfully tagged cine-backend-img:latest
laura@ubuntuserver:~/despliegue-multicapa-linux$
```

Descripción: Se compila con éxito la imagen del Backend mediante docker build asignándole el tag cine-backend-img:latest. Se verifica en el registro de la consola la finalización exitosa del empaquetado del archivo app.jar y la creación de la imagen final basada en Alpine.

Evidencia Backend funcionando:

```
laura@ubuntuserver:~$ cd ~/despliegue-multicapa-linux
docker compose up -d
[+] Running 4/4
✓Container cine-db      Healthy      3.4s
✓Container cine-backend Running      0.0s
✓Container cine-frontend Running      0.0s
✓Container cine-liquibase Exited          33.8s
```



```
{
  "movies": [
    {
      "movieId": "b2c3d4e5-0001-0001-0001-000000000001",
      "title": "Inception",
      "genre": "Sci-Fi",
      "director": "Christopher Nolan",
      "releaseYear": 2010,
      "durationMin": 148,
      "stock": 3,
      "createdAt": "2026-08-30T06:46:16.457306Z",
      "updatedAt": "2026-08-30T06:46:16.457306Z"
    },
    {
      "movieId": "b2c3d4e5-0001-0001-0001-000000000002",
      "title": "The Dark Knight",
      "genre": "Action",
      "director": "Christopher Nolan",
      "releaseYear": 2008,
      "durationMin": 152,
      "stock": 2,
      "createdAt": "2026-08-30T06:46:16.457306Z",
      "updatedAt": "2026-08-30T06:46:16.457306Z"
    },
    {
      "movieId": "b2c3d4e5-0001-0001-0001-000000000003",
      "title": "Interstellar",
      "genre": "Sci-Fi",
      "director": "Christopher Nolan",
      "releaseYear": 2014,
      "durationMin": 169,
      "stock": 4,
      "createdAt": "2026-08-30T06:46:16.457306Z",
      "updatedAt": "2026-08-30T06:46:16.457306Z"
    },
    {
      "movieId": "b2c3d4e5-0001-0001-0001-000000000004",
      "title": "The Shawshank Redemption",
      "genre": "Drama",
      "director": "Frank Darabont",
      "releaseYear": 1994,
      "durationMin": 143,
      "stock": 1,
      "createdAt": "2026-08-30T06:46:16.457306Z",
      "updatedAt": "2026-08-30T06:46:16.457306Z"
    }
  ]
}
```

Descripción: Se evidencia el encendido exitoso de los contenedores mediante la terminal. Adicionalmente, se valida el funcionamiento del backend accediendo desde el navegador web (puerto 8080), el cual retorna correctamente la lista de películas en formato JSON. Esto demuestra que la API está encendida, se comunica con la base de datos y responde al exterior sin errores.

Actividad 10 — Despliegue de la base de datos

Seleccionar y desplegar un sistema gestor de base de datos, la solución deberá incluir:

- * Imagen Docker.

- * Contenedor.
- * Usuario.
- * Contraseña.
- * Base de datos.
- * Volumen.
- * Red Docker.

El motor utilizado dependerá de la tecnología seleccionada.

Evidencia Contenedor de base de datos funcionando:

```
laura@ubuntuserver:~/despliegue-multicapa-linux$ docker ps --filter "name=cine-db"
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
PORTS         NAMES
c3311a64b7f2   postgres:17   "docker-entrypoint.s..." 36 minutes ago Up 14 minutes (health
y) 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp cine-db
```

Descripción: Se comprueba mediante la terminal que el contenedor de la base de datos PostgreSQL (cine-db) está encendido y funciona correctamente (estado healthy). Además, se verifica que expone exitosamente el puerto 5432 para recibir conexiones.

Evidencia Base de datos creada:

```
laura@ubuntuserver:~/despliegue-multicapa-linux$ docker exec -it cine-db psql -U cine_user -
d cine_db -c "\dt"
          List of relations
Schema |          Name          | Type  | Owner
-----+-----+-----+-----
public | databasechangelog      | table | cine_user
public | databasechangeloglock | table | cine_user
(2 rows)
```

Descripción: Se ejecuta un comando interno (psql) dentro del contenedor para consultar la base de datos. El resultado demuestra que la base de datos fue creada con éxito y que el usuario configurado tiene acceso a las tablas generadas.

Evidencia Volumen asociado a la base de datos:

```

laura@ubuntu:~/despliegue-multicapa-linux$ docker volume ls | grep cine_data
local        despliegue-multicapa-linux_cine_data
laura@ubuntu:~/despliegue-multicapa-linux$ docker volume inspect despliegue-multicapa-
linux_cine_data
[
  {
    "CreatedAt": "2026-08-30T06:19:07Z",
    "Driver": "local",
    "Labels": {
      "com.docker.compose.config-hash": "46f079425ce42e1839961b551884c84042c72e9efc5e0
2f6870799fa879a4980",
      "com.docker.compose.project": "despliegue-multicapa-linux",
      "com.docker.compose.version": "2.40.3",
      "com.docker.compose.volume": "cine_data"
    },
    "Mountpoint": "/var/lib/docker/volumes/despliegue-multicapa-linux_cine_data/_data",
    "Name": "despliegue-multicapa-linux_cine_data",
    "Options": null,
    "Scope": "local"
  }
]

```

Descripción: Se visualiza la creación del volumen persistente en Docker (cine_data) y sus detalles técnicos. Este volumen garantiza que los datos de la base de datos se guarden físicamente en el servidor y no se pierdan al apagar o reiniciar el contenedor.

Actividad 11 — Comunicación Backend–Base de datos

Configurar el backend para conectarse correctamente con la base de datos, la comunicación deberá realizarse mediante la red Docker.

Ejemplo:

BACKEND

|

| red Docker



DATABASE

...

El aprendiz deberá demostrar que:

* El backend puede conectarse a la base de datos.

- * Puede realizar operaciones de lectura.
- * Puede realizar operaciones de escritura.
- * Los datos permanecen almacenados.

Evidencia Configuración de conexión:

```
laura@ubuntuuserver:~/despliegue-multicapa-linux$ cat ~/despliegue-multicapa-linux/docker-compose.yml | grep -A 5 "DB_URL"
- DB_URL=jdbc:postgresql://postgres:5432/cine_db
- DB_USER=cine_user
- DB_PASSWORD=cine_password
depends_on:
  postgres:
    condition: service_healthy
```

Descripción: Se visualiza la configuración en el archivo docker-compose.yml donde se le pasa al backend la URL de la base de datos, el usuario y la contraseña. Al usar la palabra postgres en la URL, se demuestra que el backend se conecta directamente a través de la red interna de Docker.

Evidencia Operación de lectura:

```
laura@ubuntuuserver:~/despliegue-multicapa-linux$ curl -X GET http://localhost:8080/api/v1/movies
[{"movieId":"b2c3d4e5-0001-0001-0001-000000000001","title":"Inception","genre":"Sci-Fi","director":"Christopher Nolan","releaseYear":2010,"durationMin":148,"stock":3,"createdAt":"2026-08-30T06:46:16.457306Z","updatedAt":"2026-08-30T06:46:16.457306Z"}, {"movieId":"b2c3d4e5-0001-0001-0001-000000000002","title":"The Dark Knight","genre":"Action","director":"Christopher Nolan","releaseYear":2008,"durationMin":152,"stock":2,"createdAt":"2026-08-30T06:46:16.457306Z","updatedAt":"2026-08-30T06:46:16.457306Z"}, {"movieId":"b2c3d4e5-0001-0001-0001-000000000003","title":"Interstellar","genre":"Sci-Fi","director":"Christopher Nolan","releaseYear":2014,"durationMin":169,"stock":4,"createdAt":"2026-08-30T06:46:16.457306Z","updatedAt":"2026-08-30T06:46:16.457306Z"}, {"movieId":"b2c3d4e5-0001-0001-0001-000000000004","title":"The Shawshank Redemption","genre":"Drama","director":"Frank Darabont","releaseYear":1994,"durationMin":142,"stock":1,"createdAt":"2026-08-30T06:46:16.457306Z","updatedAt":"2026-08-30T06:46:16.457306Z"}, {"movieId":"b2c3d4e5-0001-0001-0001-000000000005","title":"Pulp Fiction","genre":"Thriller","director":"Quentin Tarantino","releaseYear":1994,"durationMin":154,"stock":2,"createdAt":"2026-08-30T06:46:16.457306Z","updatedAt":"2026-08-30T06:46:16.457306Z"}]laura@ubuntuserver:~/despliegue-multicapa-linux$
```

Descripción: Se realiza una operación de lectura enviando una petición GET al backend. El sistema recibe la solicitud, se comunica internamente con la base de datos para extraer la información, y retorna con éxito la lista de películas en formato JSON.

Evidencia Operación de escritura:

```
ovies \~/despliegue-multicapa-linux$ curl -X POST http://localhost:8080/api/v1/movies \
-H "Content-Type: application/json" \
-d '{"title":"The Matrix","genre":"Sci-Fi","director":"Lana Wachowski","releaseYear":1999,"durationMin":136,"stock":10}'
{"movieId":"19c4d4d2-a7a1-48c9-bbf0-3e1f0412497e","title":"The Matrix","genre":"Sci-Fi","director":"Lana Wachowski","releaseYear":1999,"durationMin":136,"stock":10,"createdAt":null,"updatedAt":null}
```

Descripción: Se realiza una operación de escritura (POST) enviando los datos de una nueva película. El backend recibe el JSON, lo procesa y lo inserta permanentemente en la base de datos, retornando el registro exitoso con su nuevo identificador único asignado.

Evidencia Logs mostrando comunicación exitosa:

```
datedAt":null}}laura@ubuntu:~/despliegue-muldocker$ compose logs backend | grep -i "pool"
"grep -i "pool"
cine-backend | 2026-08-30T06:46:54.862Z INFO 1 --- [cinema] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
cine-backend | 2026-08-30T06:46:58.591Z INFO 1 --- [cinema] [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection org.postgresql.jdbc.PgConnection@356f20b7
cine-backend | 2026-08-30T06:46:58.602Z INFO 1 --- [cinema] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
cine-backend | 2026-08-30T06:47:01.585Z INFO 1 --- [cinema] [main] org.hibernate.orm.connections.pooling : HHH10001005: Database info:
cine-backend | Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-1)']
cine-backend | Minimum pool size: undefined/unknown
cine-backend | Maximum pool size: undefined/unknown
cine-backend | 2026-08-30T07:02:18.682Z INFO 1 --- [cinema] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
cine-backend | 2026-08-30T07:02:20.746Z INFO 1 --- [cinema] [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection org.postgresql.jdbc.PgConnection@356f20b7
cine-backend | 2026-08-30T07:02:20.785Z INFO 1 --- [cinema] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
cine-backend | 2026-08-30T07:02:22.130Z INFO 1 --- [cinema] [main] org.hibernate.orm.connections.pooling : HHH10001005: Database info:
cine-backend | Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-1)']
cine-backend | Minimum pool size: undefined/unknown
cine-backend | Maximum pool size: undefined/unknown
cine-backend | 2026-08-30T12:12:54.649Z WARN 1 --- [cinema] [l-1:housekeeper] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Thread starvation or clock leap detected (housekeeper delta=3h40m6s510ms203µs677ns).
cine-backend | 2026-08-30T12:13:30.992Z WARN 1 --- [cinema] [l-1:housekeeper] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Thread starvation or clock leap detected (housekeeper delta=9m3s61ms738µs463ns).
```

Descripción: Se analizan los registros (logs) del backend, confirmando la inicialización del pool de conexiones (HikariCP) y la creación exitosa de la conexión directa con PostgreSQL. Esto valida que la comunicación entre ambos contenedores a través de la red interna es estable y sin errores.

Actividad 12 — Comunicación Frontend-Backend

Configurar el frontend para consumir la API o servicios proporcionados por el backend.

Ejemplo:

```text

FRONTEND

|

| HTTP / REST



BACKEND

|



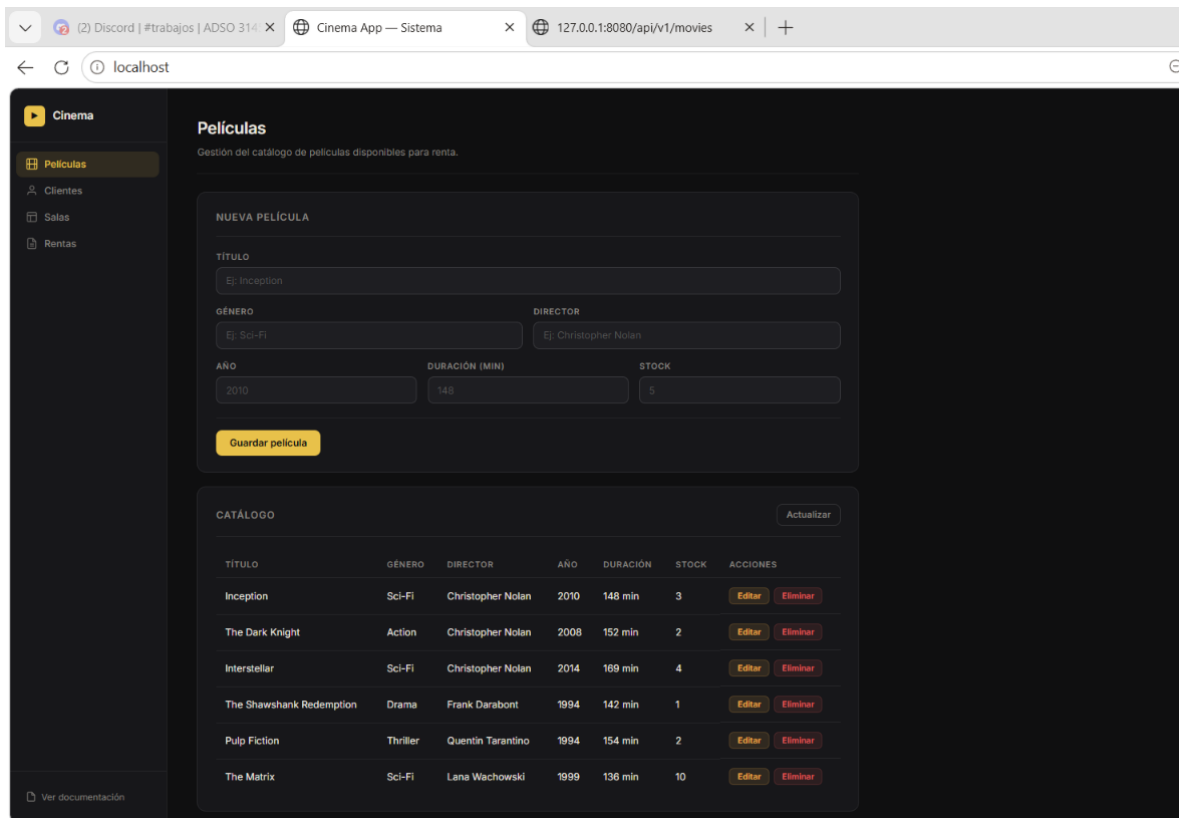
DATABASE

...

El frontend deberá permitir como mínimo:

- \* Consultar información.
- \* Registrar información.
- \* Actualizar información o realizar otra operación equivalente.
- \* Mostrar respuestas del backend.

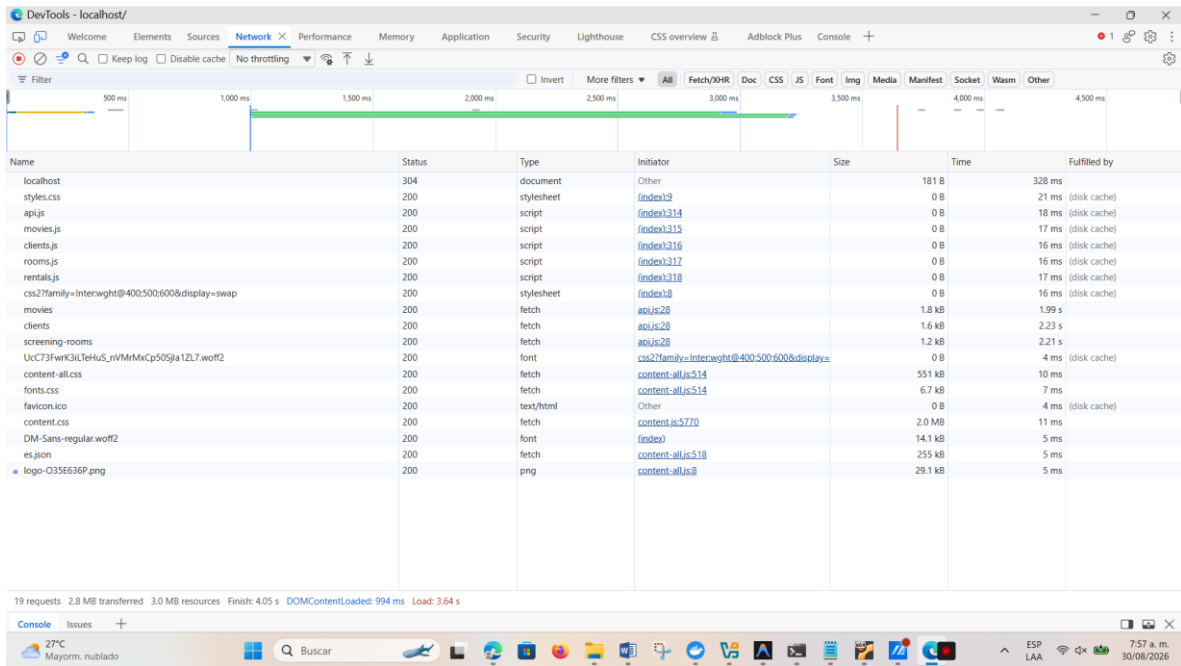
**Evidencia Frontend funcionando:**



**Descripción:** Interfaz gráfica (Frontend) cargada exitosamente en el navegador anfitrión accediendo a través del puerto 80 (localhost). Se evidencia que el contenedor web funciona de manera correcta, cargando los estilos visuales y la estructura de la aplicación.

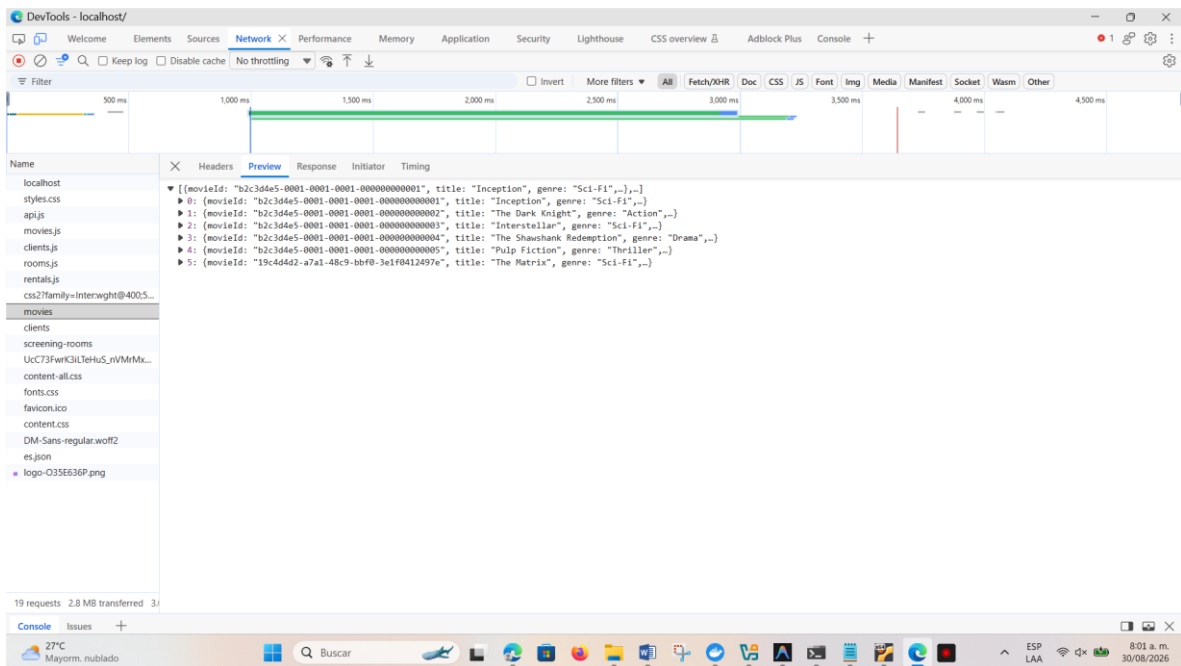
---

**Evidencia Solicitud del frontend hacia el backend:**



**Descripción:** Inspección de red (Network) del navegador donde se evidencia la petición HTTP (tipo fetch) realizada de forma exitosa por el Frontend hacia el punto de enlace (movies) expuesto por el Backend, comprobando el puente de comunicación entre ambas capas.

## Evidencia Respuesta recibida desde el backend:



**Descripción:** Inspección de red mostrando la vista previa de la respuesta (Preview) entregada por el Backend. Se confirma que los datos de las películas viajan correctamente en formato JSON desde la

base de datos hasta el navegador, donde el Frontend los procesa para mostrarlos en la interfaz gráfica.

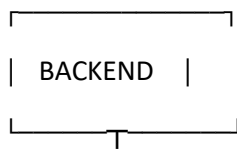
### Actividad 13 — Integración de la aplicación

Integrar los tres componentes:

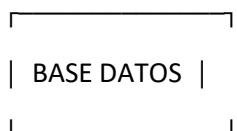
```
```text
```



| HTTP



|



```
```
```

La aplicación deberá funcionar como un único sistema.

**Evidencia Aplicación completa funcionando:**



Cinéma

Películas

Cientes

Salas

Rentas

Ver documentación

## Películas

Gestión del catálogo de películas disponibles para renta.

NUEVA PELÍCULA

TÍTULO

Avatar

GÉNERO

Sci-Fi

DIRECTOR

James Cameron

AÑO

2009

DURACIÓN (MIN)

162

STOCK

5

Guardar película

CATÁLOGO

Actualizar

| TÍTULO                   | GÉNERO   | DIRECTOR          | AÑO  | DURACIÓN | STOCK | ACCIONES                              |
|--------------------------|----------|-------------------|------|----------|-------|---------------------------------------|
| Inception                | Sci-Fi   | Christopher Nolan | 2010 | 148 min  | 3     | <div>Editar</div> <div>Eliminar</div> |
| The Dark Knight          | Action   | Christopher Nolan | 2008 | 152 min  | 2     | <div>Editar</div> <div>Eliminar</div> |
| Interstellar             | Sci-Fi   | Christopher Nolan | 2014 | 169 min  | 4     | <div>Editar</div> <div>Eliminar</div> |
| The Shawshank Redemption | Drama    | Frank Darabont    | 1994 | 142 min  | 1     | <div>Editar</div> <div>Eliminar</div> |
| Pulp Fiction             | Thriller | Quentin Tarantino | 1994 | 154 min  | 2     | <div>Editar</div> <div>Eliminar</div> |
| The Matrix               | Sci-Fi   | Lana Wachowski    | 1999 | 136 min  | 10    | <div>Editar</div> <div>Eliminar</div> |

Cinéma

Películas

Cientes

Salas

Rentas

Ver documentación

NUEVA PELÍCULA

TÍTULO

Ej: Inception

GÉNERO

Ej: Sci-Fi

DIRECTOR

Ej: Christopher Nolan

AÑO

2010

DURACIÓN (MIN)

148

STOCK

5

Guardar película

CATÁLOGO

Actualizar

| TÍTULO                   | GÉNERO   | DIRECTOR          | AÑO  | DURACIÓN | STOCK | ACCIONES                              |
|--------------------------|----------|-------------------|------|----------|-------|---------------------------------------|
| Inception                | Sci-Fi   | Christopher Nolan | 2010 | 148 min  | 3     | <div>Editar</div> <div>Eliminar</div> |
| The Dark Knight          | Action   | Christopher Nolan | 2008 | 152 min  | 2     | <div>Editar</div> <div>Eliminar</div> |
| Interstellar             | Sci-Fi   | Christopher Nolan | 2014 | 169 min  | 4     | <div>Editar</div> <div>Eliminar</div> |
| The Shawshank Redemption | Drama    | Frank Darabont    | 1994 | 142 min  | 1     | <div>Editar</div> <div>Eliminar</div> |
| Pulp Fiction             | Thriller | Quentin Tarantino | 1994 | 154 min  | 2     | <div>Editar</div> <div>Eliminar</div> |
| The Matrix               | Sci-Fi   | Lana Wachowski    | 1999 | 136 min  | 10    | <div>Editar</div> <div>Eliminar</div> |
| Avatar                   | Sci-Fi   | James Cameron     | 2009 | 162 min  | 5     | <div>Editar</div> <div>Eliminar</div> |

**Descripción:** Prueba de integración del sistema multicapa. Se evidencia el funcionamiento integral de la aplicación mediante la creación de un nuevo registro ("Avatar") desde el Frontend, el cual viaja a través del Backend y persiste exitosamente en la base de datos, mostrándose reflejado en tiempo real en la vista del catálogo.

## **Actividad 14 — Docker Compose**

**Crear un archivo:** docker-compose.yml

El archivo deberá permitir levantar la solución completa, como mínimo deberá contener:

Frontend

Backend

Base de datos

Redes

Volúmenes

**Ejecutar:** docker compose up -d

**Comprobar:** docker compose ps

**Evidencia Archivo Docker Compose:**

```

laura@ubuntuuserver: ~/despl X + v
laura@ubuntuuserver:~/desplieque-multicapa-linux$ cat ~/desplieque-multicapa-linux/docker-compose.yml
services:
----- FRONTEND -----
frontend:
 build:
 context: ./Frontend_Cine/cinema
 container_name: cine-frontend
 ports:
 - "80:80"
 depends_on:
 - backend
 networks:
 - cine_network
 restart: unless-stopped

----- BACKEND -----
backend:
 build:
 context: ./Backend_API_Cine/cinema
 container_name: cine-backend
 ports:
 - "8080:8080"
 environment:
 - SPRING_PROFILES_ACTIVE=staging
 - DB_URL=jdbc:postgresql://postgres:5432/cine_db
 - DB_USER=cine_user
 - DB_PASSWORD=cine_password
 depends_on:
 postgres:
 condition: service_healthy
 liquibase:
 condition: service_completed_successfully
 networks:
 - cine_network
 restart: unless-stopped

----- BASE DE DATOS -----
postgres:
 image: postgres:17
 container_name: cine-db
 environment:

```

```

laura@ubuntuuserver: ~/despl X + v
 POSTGRES_DB: cine_db
 POSTGRES_USER: cine_user
 POSTGRES_PASSWORD: cine_password
 ports:
 - "5432:5432"
 healthcheck:
 test: ["CMD-SHELL", "pg_isready -U cine_user -d cine_db"]
 interval: 5s
 timeout: 5s
 retries: 10
 volumes:
 - cine_data:/var/lib/postgresql/data
 networks:
 - cine_network
 restart: unless-stopped

----- LIQUIBASE (MIGRACIONES) -----
liquibase:
 build:
 context: ./BaseDeDatos_Cine/cinema/docker/liquibase
 container_name: cine-liquibase
 depends_on:
 postgres:
 condition: service_healthy
 volumes:
 - ./BaseDeDatos_Cine/cinema:/liquibase/changelog
 command:
 - --search-path=/liquibase/changelog
 - --changelogFile=changelog-master.yaml
 - --url=jdbc:postgresql://postgres:5432/cine_db
 - --username=cine_user
 - --password=cine_password
 - update
 networks:
 - cine_network
 restart: on-failure

networks:
 cine_network:
 driver: bridge

```

```
volumes:
 cine_data:
laura@ubuntuserver:~/despliegue-multicapa-linux$
```

**Descripción:** Archivo docker-compose.yml de la solución. Se evidencia la declaración de todos los servicios requeridos (Frontend, Backend, Base de Datos y motor de migraciones), así como la configuración centralizada de la red interna (cine\_network) y los volúmenes persistentes.

---

**Evidencia todos los servicios funcionando:**

```
laura@ubuntuserver:~/despliegue-multicapa-linux$ docker compose ps
NAME IMAGE COMMAND SERVICE CR
cine-backend despliegue-multicapa-linux-backend "java -jar app.jar" backend 7
cine-db postgres:17 "docker-entrypoint.sh" postgres 7
cine-frontend despliegue-multicapa-linux-frontend "/docker-entrypoint.sh" frontend 7
```

**Descripción:** Se ejecuta el comando docker compose ps para comprobar el estado general de los servicios. El resultado demuestra que los contenedores del Frontend, Backend y Base de Datos se encuentran funcionando simultáneamente de forma estable (estado Up), cumpliendo con la orquestación completa de la solución.

## Actividad 15 — Prueba de recuperación

**Detener los servicios:** docker compose stop

Comprobar su estado.

Posteriormente:

docker compose start

Realizar una segunda prueba utilizando:

docker compose down

docker compose up -d

Finalmente, reconstruir:

docker compose up -d --build

#### Evidencia Servicios detenidos:

```
laura@ubuntu:~/despliegue-multicapa-linux$ docker compose stop
[+] Stopping 4/4
✓Container cine-frontend Stopped 2.0s
✓Container cine-backend Stopped 2.9s
✓Container cine-liquibase Stopped 0.0s
✓Container cine-db Stopped 0.7s
laura@ubuntu:~/despliegue-multicapa-linux$ docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
laura@ubuntu:~/despliegue-multicapa-linux$
```

**Descripción:** Se ejecuta el comando docker compose stop deteniendo todos los contenedores de forma segura. Posteriormente, se valida con docker compose ps que la lista está vacía, confirmando que no existe ningún servicio activo en la solución en este momento.

#### Evidencia Servicios recuperados:

```
laura@ubuntu:~/despliegue-multicapa-linux$ docker compose start
[+] Running 4/4
✓Container cine-db Healthy 8.2s
✓Container cine-liquibase Exited 11.1s
✓Container cine-backend Started 0.5s
✓Container cine-frontend Started 0.9s
laura@ubuntu:~/despliegue-multicapa-linux$ docker compose ps
NAME IMAGE COMMAND SERVICE CR
EATED STATUS PORTS
cine-backend despliegue-multicapa-linux-backend "java -jar app.jar" backend 7
hours ago Up 6 seconds 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
cine-db postgres:17 "docker-entrypoint.s..." postgres 7
hours ago Up 24 seconds (healthy) 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
cine-frontend despliegue-multicapa-linux-frontend "/docker-entrypoint..." frontend 7
hours ago Up 5 seconds 0.0.0.0:80->80/tcp, [::]:80->80/tcp
laura@ubuntu:~/despliegue-multicapa-linux$
```

**Descripción:** Se reanuda la operación con el comando docker compose start. El orquestador levanta nuevamente los contenedores pausados y se verifica (mediante el comando ps) que los servicios regresan a su estado activo ("Up") sin presentar problemas, validando la correcta recuperación del sistema.

#### Evidencia Servicios reconstruidos:

```
laura@ubuntu:~/despliegue-multicapa-linux$ docker compose down
[+] Running 5/5
✓Container cine-frontend Remov... 0.8s
✓Container cine-backend Remove... 1.1s
✓Container cine-liquibase Remo... 0.1s
✓Container cine-db Removed 1.2s
✓Network despliegue-multicapa-linux_cine_network Removed 0.3s
```

```

laura@ubuntu-server:~/despliegue-multicapa-linux$ docker compose up -d --build
WARN[0000] Docker Compose is configured to build using Bake, but buildx isn't installed
[+] Building 128.8s (32/32) FINISHED docker:default
=> [liquibase internal] load build definition from Dockerfile 0.1s
=> => transferring dockerfile: 175B 0.0s
=> [liquibase internal] load metadata for docker.io/liquibase/liquibase:4.27 1.8s
=> [liquibase internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [liquibase 1/2] FROM docker.io/liquibase/liquibase:4.27@sha256:141a9fb5df99e43be4 0.1s
=> => resolve docker.io/liquibase/liquibase:4.27@sha256:141a9fb5df99e43be49e507dc189 0.1s
=> CACHED [liquibase 2/2] RUN lpm add postgresql --global 0.0s
=> [liquibase] exporting to image 0.5s
=> => exporting layers 0.0s
=> => exporting manifest sha256:da532857a21fb8a62b2c9764c07b539ea4cccd187b46581f8e 0.0s
=> => exporting config sha256:43eda49c739dd3a3e644fd7bdcae6b47ca585b684f0e6f1e46f2ec 0.0s
=> => exporting attestation manifest sha256:2f2d778036ba0374916a41829420e930c784bda3 0.1s
=> => exporting manifest list sha256:ee04fe6591744b43bf0d52a9ab75b6fc32b5524bc061635 0.1s
=> => naming to docker.io/library/despliegue-multicapa-linux-liquibase:latest 0.0s
=> => unpacking to docker.io/library/despliegue-multicapa-linux-liquibase:latest 0.0s
=> [liquibase] resolving provenance for metadata file 0.0s
=> [backend internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 601B 0.0s
=> [backend internal] load metadata for docker.io/library/eclipse-temurin:17-jre-alp 0.8s
=> [backend internal] load metadata for docker.io/library/maven:3.9.6-eclipse-temuri 0.7s
=> [backend internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [backend builder 1/6] FROM docker.io/library/maven:3.9.6-eclipse-temurin-17@sha25 0.2s
=> => resolve docker.io/library/maven:3.9.6-eclipse-temurin-17@sha256:29a1658b1f3078 0.1s
=> [backend stage-1 1/3] FROM docker.io/library/eclipse-temurin:17-jre-alpine@sha256 0.2s
=> => resolve docker.io/library/eclipse-temurin:17-jre-alpine@sha256:27cc8849148c0fd 0.1s
=> [backend internal] load build context 0.2s
=> => transferring context: 56.95kB 0.1s
=> CACHED [backend builder 2/6] WORKDIR /app 0.0s
=> [backend builder 3/6] COPY pom.xml . 0.4s
=> [backend builder 4/6] RUN mvn dependency:go-offline -B 82.5s
=> [backend builder 5/6] COPY src ./src 0.3s
=> [backend builder 6/6] RUN mvn clean package -DskipTests -B 28.9s
=> CACHED [backend stage-1 2/3] WORKDIR /app 0.0s
=> [backend stage-1 3/3] COPY --from=builder /app/target/*.jar app.jar 0.4s
=> [backend] exporting to image 4.4s
=> => exporting layers 3.5s
=> => exporting manifest sha256:bb9aeeca3992c71bcc07a9eac12c06608899ce91f7ae2aa0573f1 0.1s
=> => exporting config sha256:0f59cd8c923a3ealf2e6102c7e0c3faec8aef6a35c142b788d2b55 0.0s
=> => exporting attestation manifest sha256:9e9a8381b4e58f01bf97e6e09735e60f87227abf 0.1s
=> => exporting manifest list sha256:856f9ae30bb4ad0a14ce0ce3e3d1bab1172c8db618e2c50 0.0s

```

```

=> => naming to docker.io/library/despliegue-multicapa-linux-backend:latest 0.0s
=> => unpacking to docker.io/library/despliegue-multicapa-linux-backend:latest 0.5s
=> [backend] resolving provenance for metadata file 0.0s
=> [frontend internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 283B 0.0s
=> [frontend internal] load metadata for docker.io/library/nginx:alpine 1.0s
=> [frontend internal] load .dockerignore 0.1s
=> => transferring context: 2B 0.0s
=> CACHED [frontend 1/3] FROM docker.io/library/nginx:alpine@sha256:db35bfc6b2951e7f 0.2s
=> => resolve docker.io/library/nginx:alpine@sha256:db35bfc6b2951e7f8a72db5db120288c 0.1s
=> [frontend internal] load build context 0.1s
=> => transferring context: 62.45kB 0.0s
=> [frontend 2/3] COPY nginx.conf /etc/nginx/conf.d/default.conf 0.4s
=> [frontend 3/3] COPY . /usr/share/nginx/html 0.2s
=> [frontend] exporting to image 1.1s
=> => exporting layers 0.5s
=> => exporting manifest sha256:625b0544735a984c2ceaa96fe95ed847d83e04c29b70892ed2c8 0.1s
=> => exporting config sha256:c8c66f9cecfa03bee8aafb9a7bd4eef052159221e1900912470738 0.0s
=> => exporting attestation manifest sha256:5fe8b99c41051c6595999d8ae832a4f07908f42e 0.1s
=> => exporting manifest list sha256:ec2fb19c8444707c503cec586b86f937b5756eb0a826a8 0.1s
=> => naming to docker.io/library/despliegue-multicapa-linux-frontend:latest 0.0s
=> => unpacking to docker.io/library/despliegue-multicapa-linux-frontend:latest 0.2s
=> [frontend] resolving provenance for metadata file 0.0s
[+] Running 8/8
✔ backend Built 0.0s
✔ frontend Built 0.0s
✔ liquibase Built 0.0s
✔ Network despliegue-multicapa-linux_cine_network Created 0.4s
✔ Container cine-db Healthy 7.9s
✔ Container cine-liquibase Exit... 13.7s
✔ Container cine-backend Start... 13.9s
✔ Container cine-frontend Start... 14.5s
laura@ubuntu-server:~/despliegue-multicapa-linux$

```

**Descripción:** Prueba de recuperación extrema. Se evidencia la destrucción total de los contenedores y la red mediante el comando `docker compose down`. Posteriormente, se realiza una reconstrucción completa de las imágenes desde el código fuente utilizando la bandera `--build`. El orquestador vuelve a levantar la solución desde cero de forma exitosa, demostrando que la infraestructura es resiliente, inmutable y que los datos están a salvo.

## Actividad 16 — Comprobación desde la intranet

Obtener la IP de la máquina virtual: `ip addr`

Desde el equipo anfitrión acceder al frontend:

```
```text
```

```
http://IP_MAQUINA_VIRTUAL:PUERTO
```

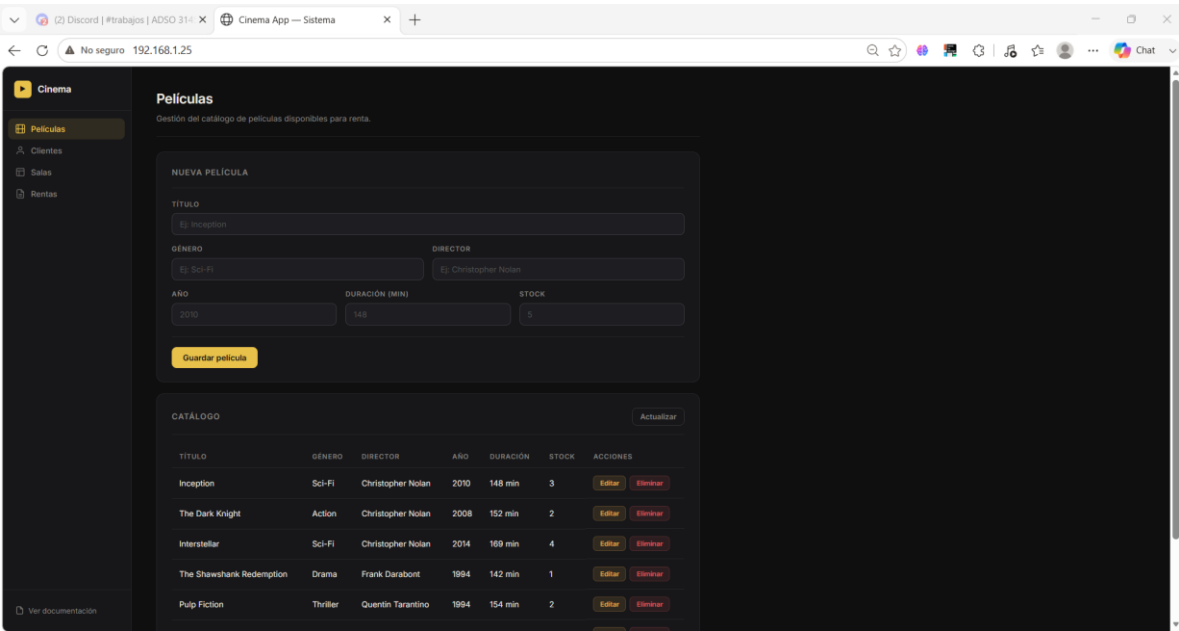
```
```
```



Si la red de la máquina virtual está configurada como puente, realizar la prueba desde otro equipo conectado a la misma red.

Evidencia Aplicación funcionando desde el equipo anfitrión:

```
laura@ubuntu-server:~/despliegue-multicapa-linux$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
 link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
 inet 127.0.0.1/8 scope host lo
 valid_lft forever preferred_lft forever
 inet6 ::1/128 scope host noprefixroute
 valid_lft forever preferred_lft forever
2: enp8s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
 link/ether 08:00:27:6d:1c:a7 brd ff:ff:ff:ff:ff:ff
 altname enx0800276d1ca7
 inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp8s3
 valid_lft 86118sec preferred_lft 86118sec
 inet6 fd17:02c:f037:2:a80:27ff:fedd:1ca7/64 scope global dynamic mngtmpaddr noprefixroute
 valid_lft 86118sec preferred_lft 14118sec
 inet6 fe80::a00:27ff:fe6d:1ca7/64 scope link proto kernel_ll
 valid_lft forever preferred_lft forever
4: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
 link/ether aa:ae:62:11:28:e4:86 brd ff:ff:ff:ff:ff:ff
 inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
 valid_lft forever preferred_lft forever
 inet6 fe80::a8e6:21ff:fe28:e486/64 scope link proto kernel_ll
 valid_lft forever preferred_lft forever
5: br-d6d2d37c61f8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
 link/ether 96:c1:46:28:00:bb brd ff:ff:ff:ff:ff:ff
 inet 172.18.0.1/16 brd 172.18.255.255 scope global br-d6d2d37c61f8
 valid_lft forever preferred_lft forever
 inet6 fe80::94c1:46ff:fe28:bb/64 scope link proto kernel_ll
 valid_lft forever preferred_lft forever
7: veth234d0b9e1f2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
 link/ether ea:2e:ba:85:ec:72 brd ff:ff:ff:ff:ff:ff link-netnsid 1
 inet6 fe80::e82e:baff:fe05:ec72/64 scope link proto kernel_ll
 valid_lft forever preferred_lft forever
9: veth139ea31e1f2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-d6d2d37c61f8 state UP group default
 link/ether 8a:a0:28:f1:bd:13 brd ff:ff:ff:ff:ff:ff link-netnsid 3
 inet6 fe80::8a:a0:28ff:fe1:bd13/64 scope link proto kernel_ll
 valid_lft forever preferred_lft forever
28: br-39e3d2e07748: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
 link/ether fe:b4:54:ea:ed:2d brd ff:ff:ff:ff:ff:ff
 inet 172.19.0.1/16 brd 172.19.255.255 scope global br-39e3d2e07748
 valid_lft forever preferred_lft forever
 inet6 fe80::fcb4:54ff:feea:ed2d/64 scope link proto kernel_ll
 valid_lft forever preferred_lft forever
29: vethc38ac8c1f2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-39e3d2e07748 state UP group default
 link/ether 06:55:86:4b:0c:8b brd ff:ff:ff:ff:ff:ff link-netnsid 0
 inet6 fe80::4b5:86ff:fe0b:ca/64 scope link proto kernel_ll
 valid_lft forever preferred_lft forever
32: veth6491ecf1f2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-39e3d2e07748 state UP group default
 link/ether 66:52:fb:5f:71:b8 brd ff:ff:ff:ff:ff:ff link-netnsid 2
 inet6 fe80::6452:fbff:fe5f:71b8/64 scope link proto kernel_ll
 valid_lft forever preferred_lft forever
33: veth2858ac9e1f2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-39e3d2e07748 state UP group default
 link/ether 06:34:48:a5:e8:57 brd ff:ff:ff:ff:ff:ff link-netnsid 4
 inet6 fe80::434:48ff:fea5:e857/64 scope link proto kernel_ll
 valid_lft forever preferred_lft forever
laura@ubuntu-server:~/despliegue-multicapa-linux$
```



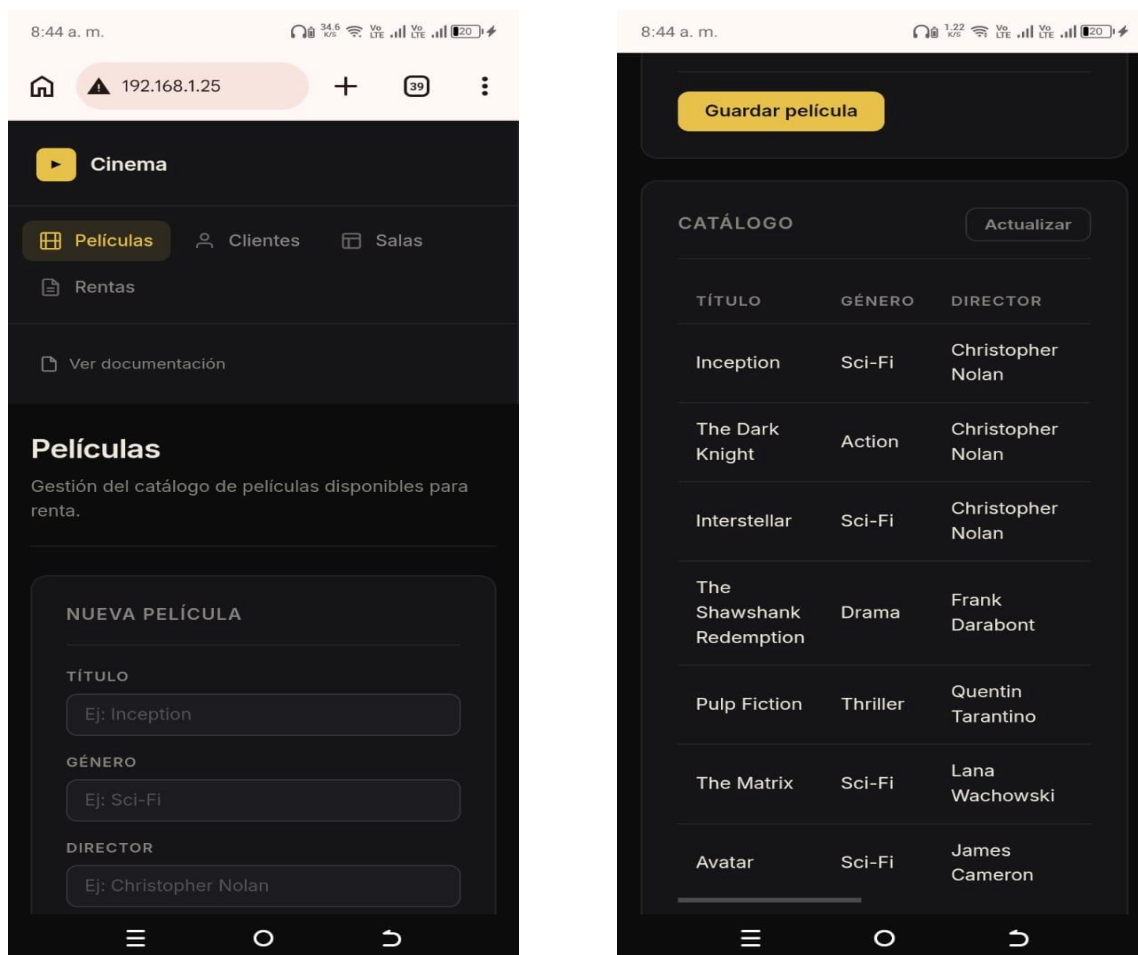
**Descripción:** Se ejecuta el comando ip addr en la máquina virtual para identificar las interfaces de red. Posteriormente, se comprueba el acceso a la aplicación desde el navegador del equipo anfitrión



apuntando a la IP de red local (192.168.1.25). La carga exitosa de la interfaz y los datos demuestra que el reenvío de puertos hacia la máquina virtual funciona correctamente.

---

#### Evidencia Aplicación funcionando desde otro equipo de la intranet:



**Descripción:** Prueba exitosa de acceso desde un cliente externo en la intranet (dispositivo móvil). Al ingresar la IP local del equipo anfitrión, se logra consumir tanto la interfaz gráfica (Frontend) como los datos (Backend/BD) a través de la red LAN. Esto valida que el sistema multicapa está completamente integrado, orquestado y accesible para usuarios externos en la misma red.

#### Actividad 17 — Documentación de la solución

**Tecnologías seleccionadas:** Frontend: HTML, CSS y Vanilla JavaScript servido mediante un servidor web Nginx (basado en Alpine Linux para optimizar peso). Backend: Java Spring Boot (compilado con Maven y ejecutado con Eclipse Temurin JDK 17). Base de Datos: PostgreSQL versión 17. Migraciones

de BD: Liquibase (utilizado para crear la estructura de tablas y relaciones de forma automatizada). Orquestación y Despliegue: Docker y Docker Compose. Entorno Anfitrión: Máquina virtual con Ubuntu Server ejecutada sobre Windows mediante VirtualBox.

---

**Arquitectura:** Se implementó un patrón de Arquitectura Multicapa (3 Tier Architecture):

Capa de Presentación (Frontend): Interfaz gráfica accesible vía HTTP que consume servicios. Capa de Lógica de Negocio (Backend): API REST que recibe las peticiones, procesa las reglas del cine y se comunica con los datos. Capa de Datos (Base de datos): Motor relacional (PostgreSQL) que almacena de forma persistente la información de películas, clientes, salas y rentas.

---

**Estructura del proyecto:** El repositorio se dividió lógicamente en tres directorios principales y un orquestador central:

/Frontend\_Cine/: Código estático y configuraciones de Nginx.

/Backend\_API\_Cine/: Código fuente de Spring Boot y pom.xml.

/BaseDeDatos\_Cine/: Archivos YAML de migraciones (changelogs) para Liquibase.

docker-compose.yml: Archivo raíz que integra todas las partes.

---

#### **Dockerfiles:**

Backend: Implementa Multi-stage build. Usa una imagen de Maven para compilar el código (mvn clean package) y luego transfiere únicamente el archivo .jar resultante a una imagen limpia de JRE, reduciendo drásticamente el peso final.

Frontend: Utiliza la imagen oficial de Nginx, copiando los estáticos al directorio público (/usr/share/nginx/html) y sobrescribiendo el archivo default.conf.

Liquibase: Descarga la dependencia del controlador JDBC de PostgreSQL mediante el administrador de paquetes lpm antes de aplicar las migraciones.

---

**Docker Compose:** Se utilizó docker-compose.yml versión 3+ para orquestar los 4 contenedores (frontend, backend, bd y migraciones). Se implementó un control de dependencias estricto mediante la directiva depends\_on: condition: service\_healthy y service\_completed\_successfully, garantizando que el backend no inicie hasta que la base de datos esté sana y las migraciones de liquibase hayan terminado.

---

**Redes:** Se creó una red interna personalizada llamada cine\_network utilizando el driver bridge. Esta red aísla los contenedores del exterior y permite la resolución de nombres DNS interna (ej. el backend se conecta a la base de datos simplemente llamando al host postgres).

---

**Volúmenes:**

Persistente (Named Volume): Se creó el volumen cine\_data anclado a la ruta /var/lib/postgresql/data del contenedor de la base de datos. Esto garantiza que la información no se pierda aunque el contenedor sea destruido.

Montajes locales (Bind Mounts): Se montó el directorio local de BaseDeDatos\_Cine hacia el contenedor de Liquibase para que éste leyera los archivos .yaml en tiempo real.

---

**Variables de entorno:** Se usaron variables inyectadas desde el Compose para asegurar los servicios sin quemar credenciales en el código:

Backend: SPRING\_PROFILES\_ACTIVE, DB\_URL (apuntando a postgres:5432), DB\_USER, DB\_PASSWORD.

Postgres: POSTGRES\_DB, POSTGRES\_USER, POSTGRES\_PASSWORD.

---

**Puertos:** La comunicación con la máquina virtual se expuso a través de los siguientes mapeos (Host:Contenedor):

Frontend web: 80:80 API REST

Backend: 8080:8080

PostgreSQL: 5432:5432

---

**Servicios:**

cine-db: Base de datos PostgreSQL.

cine-liquibase: Servicio efímero que ejecuta migraciones y se apaga (Exited).

cine-backend: API REST en Java.

cine-frontend: Servidor web Nginx

---

**Comandos utilizados:**

docker compose up -d --build: Reconstruir imágenes y levantar en segundo plano.

docker compose ps: Validar el estado de los contenedores.

docker compose stop / start / down: Control de ciclo de vida.

docker compose logs [servicio]: Auditoría y revisión de errores.

curl -X GET / POST: Realizar peticiones HTTP directas por consola.

ip addr y chmod: Comandos de red y permisos del sistema operativo base.

---

### **Problemas encontrados:**

Fallo de permisos en Liquibase: El contenedor de migraciones fallaba y se detenía prematuramente arrojando un error `ChangeLogParseException: (Permission denied)`, al intentar leer los archivos YAML montados desde el sistema Linux anfitrión.

Inaccesibilidad desde el Host Windows: Al intentar acceder desde el navegador de Windows a la máquina virtual (Ubuntu en VirtualBox) usando localhost o la IP, la conexión era rechazada y los servicios no cargaban, pese a que en Linux estaban marcados como "Up".

---

### **Soluciones implementadas:**

Solución a permisos: Se identificó que la carpeta del proyecto en Linux estaba bloqueada por políticas estrictas (`drwx-----`). Se utilizó el comando recursivo `chmod -R 755 BaseDeDatos_Cine` para otorgar permisos de lectura a grupos y otros, permitiendo al usuario interno de Docker leer los archivos y ejecutar la migración exitosamente.

Solución de red VirtualBox: Se determinó que al usar una red NAT en la máquina virtual, era obligatorio configurar el "Reenvío de puertos" (Port Forwarding). Se mapearon explícitamente los puertos 80 y 8080 del entorno Guest hacia el entorno Host. Posteriormente se reinició la máquina virtual, logrando acceso total desde Windows y desde otros dispositivos en la red LAN.

---

### **"Anexo: Enlaces y Credenciales de Acceso":**

---

#### **Accesos de la Aplicación (Servicios Web)**

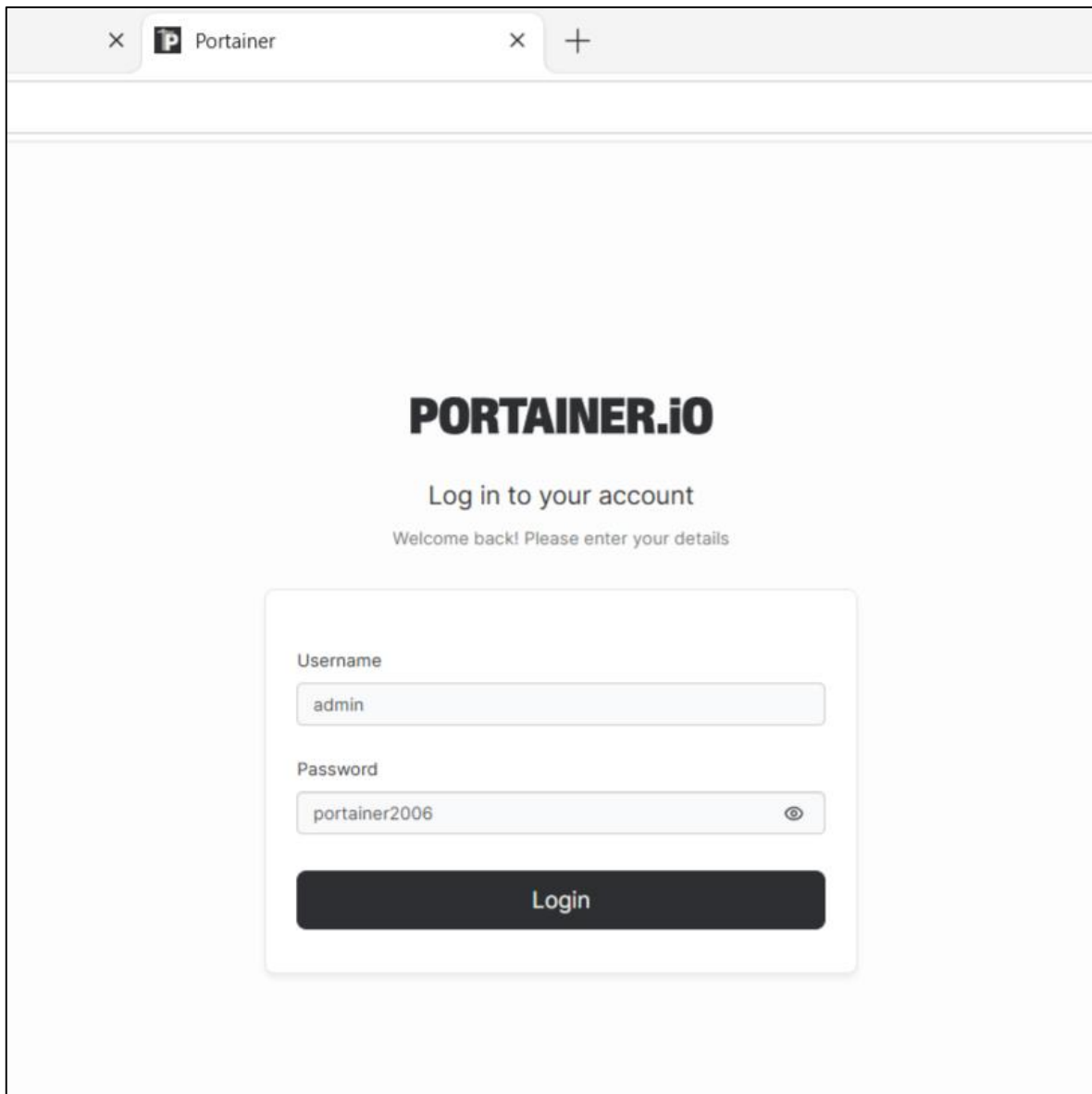
- Página Web (Frontend): Enlace Local: `http://localhost` (o <http://127.0.0.1>)
  - Enlace Intranet: <http://192.168.1.25>
  - Puerto expuesto: 80
- 

- Backend (API REST): Enlace base: <http://localhost:8080/api/v1/movies>
  - Puerto expuesto: 8080
- 

#### **Acceso a Portainer (Administración Docker)**

- Enlace: `http://localhost:9000` (o <https://localhost:9443>)
- Puerto expuesto: 9000

- Usuario: admin Contraseña: (portainer2006)



---

### Credenciales y Detalles Técnicos de la Base de Datos (PostgreSQL)

- URL de conexión externa (Para clientes como DBeaver/pgAdmin):  
postgresql://localhost:5432/cine\_db
- URL de conexión interna (Para el backend dentro de la red Docker):  
jdbc:postgresql://postgres:5432/cine\_db
- Host interno: postgres
- Puerto expuesto: 5432
- Nombre de la BD: cine\_db
- Esquema principal (Schema): cine (Nota: Las tablas no están en el esquema 'public' por seguridad arquitectónica, se estructuraron bajo el esquema 'cine' mediante Liquibase).

- Usuario: cine\_user
- Contraseña: cine\_password

**Nota para evaluación:** Si desea comprobar la persistencia de datos mediante consola interactiva, puede ejecutar `docker exec -it cine-db psql -U cine_user -d cine_db`. Una vez dentro del motor, las tablas se listan con el comando `\dt cine.*` y los registros se consultan apuntando al esquema, por ejemplo: `SELECT * FROM cine.movies;`